

Informe de auditoría: RespalDOS.GIO 5.0

Archivo auditado: RespalDOS5_0.py (2,124 líneas, MD5 a6f1ff86...) **Fecha:** 3 de octubre de 2026
Alcance: seguridad, rendimiento y funcionamiento **Dictamen:** el programa **funciona bien y respalda sin perder datos**, pero **no debe exponerse en una red que no sea de confianza** hasta corregir los hallazgos SEG-01 y SEG-02.

Importante: el archivo que subiste **no es idéntico** al que te entregué. Alguien le agregó autenticación HMAC, TLS con fijación de certificado (TOFU), verificación SHA-256 al reanudar, permisos 0600, validación de config y ruta de adb configurable. Audité **tu archivo**, no el mío.

1. Resumen ejecutivo

Área	Resultado
Funcionamiento	21 de 21 pruebas aprobadas. Un riesgo de compatibilidad sin verificar (FUN-01).
Seguridad	5 controles verificados, 7 hallazgos: 2 altos/medios de disponibilidad, 2 medios, 3 bajos.
Rendimiento	Escaneo 6x más rápido y la mitad de memoria que la 4.0. La verificación SHA-256 y el fsync cuestan entre 34% y 3x de tiempo de CPU, según el caso.

Lo bueno: la integridad de los respaldos es sólida. Cada archivo se comparó byte a byte contra el original, la reanudación detecta archivos corrompidos o truncados, y TLS + HMAC + pinning funcionan de punta a punta.

Lo urgente: el servidor de red se puede tumbar con una sola conexión de cualquier equipo de la LAN (SEG-01, SEG-02).

2. Entorno y método

Elemento	Detalle
Sistema	Linux (VM Firecracker), 1 vCPU , 3.9 GB RAM, Python 3.12.3
Herramientas	bandit, pyflakes, vermin, psutil 7.2, cryptography 46
Datos de prueba	3,020 archivos (127 MB), 20,000 archivos chicos, 100,000 archivos de 100 B, 3 archivos de 200 MB, nombres Unicode / con salto de línea / bytes inválidos
Comparativos	Versión 4.0 original, versión 5.0 que te entregué, archivo auditado
Repeticiones	Mediana de 5 corridas (3 en zip, 2 en 100k)

Pruebas registradas: 35 funcionales y de seguridad, más 30 corridas de rendimiento. Análisis estático: pyflakes sin errores; versión mínima de Python según vermin : **3.7**.

3. Hallazgos de seguridad

5. Hallazgos de seguridad

Severidad: **Alta** = explotable de forma remota con impacto real. **Media** = requiere condiciones concretas. **Baja** = endurecimiento.

ID	Sev.	Hallazgo
SEG-01	Alta	El servidor no tiene timeouts . Una conexión que no manda nada lo bloquea para siempre.
SEG-02	Media	Un solo intento con clave incorrecta termina el servidor.
SEG-03	Media	Se permite operar sin TLS y con cualquier clave , sin longitud mínima.
SEG-04	Media	La reanudación no valida las rutas del manifiesto : <code>../</code> escribe fuera del destino.
SEG-05	Baja	Un nombre de archivo con byte nulo enviado por el servidor aborta toda la descarga .
SEG-06	Baja	El prompt del cliente imprime la clave de red guardada en pantalla.
SEG-07	Info	Los respaldos salen con permisos del <code>umask</code> (carpeta 0755, archivos 0644) y sin cifrar.

Evidencia y recomendación

SEG-01: servidor sin timeouts. *Prueba:* abrí una conexión TCP y no mandé nada. Después intenté conectar un cliente legítimo con la clave correcta: falló a los 10 s y el servidor siguió bloqueado. No hay `settimeout` del lado del servidor, ni en el handshake TLS ni en la lectura de `MAGIA_RED`. *Corrección:* `conn.settimeout(NETWORK_TIMEOUT_AUTH)` justo después de `accept()`, y quitar el timeout (o subirlo) hasta que la autenticación termine.

SEG-02: una conexión mala apaga el servidor. *Prueba:* un cliente con clave equivocada fue rechazado correctamente, pero el servidor terminó su ejecución (`return`). Cualquiera en la LAN puede apagarlo antes de que llegue el cliente legítimo. *Corrección:* meter `accept()` en un ciclo; ante fallo de autenticación o handshake, cerrar esa conexión y seguir esperando (con un tope de intentos o un tiempo total).

SEG-03: sin TLS y con claves débiles. *Prueba:* con TLS desactivado, un proxy capturó el contenido de los archivos **en texto claro**. Con clave `1234` recuperé la clave **offline** desde la captura (nonce + respuesta HMAC) en **0.00 s**. Con TLS, la captura no mostró contenido legible. *Corrección:* exigir clave de al menos 12 caracteres (o solo permitir la generada), y que el modo sin TLS requiera una opción explícita y avisar en cada ejecución.

SEG-04: manifiesto sin validar. *Prueba:* escribí una entrada `../../pwn/escapó.txt` en el manifiesto de un respaldo pausado y reanudé: **el archivo se creó fuera del destino**. Requiere poder escribir en la carpeta `estados/`, por eso es Media y no Alta. El cliente de red, en cambio, sí se defiende bien (ver controles). *Corrección:* reutilizar `_ruta_segura()` en `CopiadorParalelo._copiar_uno` o al leer el manifiesto.

SEG-05: byte nulo. *Prueba:* un servidor hostil (con clave válida) envió `a\x00b.txt`. El cliente lanzó `ValueError` y perdió **todos** los archivos. *Corrección:* capturar `ValueError` en `_ruta_segura` y saltar el

archivo, drenando sus bytes.

SEG-06: clave en pantalla. *Prueba:* el prompt muestra `Enter = usar la guardada:`

`CLAVE_GUARDADA_SECRETA`. *Corrección:* mostrar solo los últimos 4 caracteres.

SEG-07: permisos de los respaldos. Es el comportamiento estándar del `umask`, pero un respaldo de documentos legible por otros usuarios del equipo suele ser indeseable. *Corrección opcional:* `chmod 0700` a la carpeta de respaldo. Los respaldos no están cifrados.

Controles verificados (funcionan)

Control	Resultado
Cliente ante servidor hostil con <code>../..</code> y ruta absoluta	Bloqueado , nada se escribe fuera del destino
Clave incorrecta	Rechazada
TLS + HMAC con <i>channel binding</i>	Transferencia íntegra (hash igual)
Fijación TOFU: si cambia el certificado	Alerta y cancela si el usuario niega
Permisos de <code>config.json</code> , llave TLS y <code>servidores_conocidos.json</code>	0600 los tres
Validación de <code>config.json</code> hostil	Valores normalizados (hilos ≤ 64 , nivel 0-9)

Análisis estático (bandit)

21 avisos, todos revisados a mano: 12 `try/except/pass` (B110), 1 `sha1` usado solo para generar un id (B324), `subprocess` con argumentos fijos o la ruta de `adb` tomada de `config.json` (B603/B607), `os.system` con cadena constante (B605) y 2 `bind` a `0.0.0.0` que piden confirmación (B104). **Ninguno es explotable por sí solo.** Lo único a vigilar es que quien pueda editar `config.json` puede hacer que se ejecute otro binario como `adb`.

4. Pruebas de funcionamiento

#	Prueba	Resultado
1	Escáner: blacklist por nombre, whitelist que gana, multi-segmento (<code>Library/Caches</code>), ignora symlinks	Aprobada
2	Respaldo general: 3,004 archivos comparados byte a byte	Aprobada, 0 diferencias
3	Sin <code>.part</code> ni estados residuales; <code>backup_info.json</code> completo	Aprobada
4	Respaldo por extensiones: 3,014 archivos sin colisión de nombres	Aprobada
5	Zip niveles 0, 6 y 9: válido, un solo <code>backup_info.json</code> , carpeta temporal borrada	Aprobada (3)

6	Pausa por Ctrl+C real (SIGINT a un proceso)	Aprobada, 0.02 s de latencia
7	Reanudar: archivos corrompidos y truncados se detectan y se vuelven a copiar	Aprobada, 3,004/3,004 correctos
8	Disco lleno (tmpfs de 3 MB real): aborta, deja pausado, limpia .part	Aprobada
9	Nombres raros: Unicode, salto de línea, bytes inválidos, vacíos	Aprobada, 7/7
10	Carpetas y archivos sin permiso (usuario nobody)	Aprobada: los reporta en backup_info.json y sigue
11	eliminar_respaldo no lista carpetas ni zips ajenos	Aprobada
12	Menú con entradas basura e inválidas (35 entradas)	Aprobada, sin Traceback
13	Modo red TLS + HMAC	Aprobada, hash idéntico
14	Config con JSON roto / valores hostiles	Aprobada

Riesgo de compatibilidad (sin verificar)

FUN-01 (Media): limpiar_terminal() imprime la secuencia ANSI \033[2J y solo usa cls si hay una excepción. En consolas de Windows que no interpretan ANSI (Windows 7/8 o cmd.exe sin modo VT) aparecerían caracteres basura en cada pantalla del menú. **No lo pude probar: no tengo Windows.** Probablemente conviene usar cls primero en Windows.

5. Pruebas de rendimiento

Mediana de corridas; hilos fijos; monitor de recursos apagado para aislar el copiado.

Prueba	4.0 original	5.0 (mía)	Auditado	Lectura
20,000 archivos chicos (tiempo)	7.93 s	6.70 s	7.62 s	Empate técnico: los rangos se traslapan
Escaneo de esos 20,000	0.43 s	0.07 s	0.07 s	6x más rápido
Memoria máxima (RSS)	72 MB	35 MB	36 MB	Mitad
3 archivos de 200 MB	0.21 s	0.29 s	0.88 s	El SHA-256 triplica el tiempo en datos en caché
Zip de 80 MB de JPG + 40 MB de texto	2.05 s	n/p	0.24 s	8x más rápido, mismo tamaño (84 MB)
100,000 archivos de 100 B	n/p	8.46 s	11.35 s	+34% por SHA-256 y fsync
Memoria con 100,000 archivos	n/p	55 MB	67 MB	Aceptable
Reanudar un respaldo ya	n/p	0.29 s	1.04 s	Revalida releyendo todo lo

n/p = no probado.

Cómo leer estos números

- **La verificación SHA-256 es una buena decisión de integridad**, pero tiene costo de CPU. Aquí es visible porque los 600 MB estaban en caché de RAM; en un USB o disco lento el dispositivo suele ser el cuello y el costo se oculta. No lo medí en un USB real ni en CPUs sin aceleración SHA. En un equipo viejo podría pesar más.
- **Reanudar ahora cuesta tanto como releer todo lo ya copiado.** En un respaldo de 200 GB interrumpido al 90%, reanudar implica hashear unos 180 GB antes de seguir. Una opción "rápida" (tamaño + fecha) para quien confíe en el destino evitaría esto.
- **El monitor de recursos ya no se frena a sí mismo.** En una medición previa con mi versión, la 4.0 pasó de 7.5 a 19.5 s con psutil y la 5.0 se mantuvo en 6.6–8.3 s. **No lo repetí sobre el archivo auditado**, aunque el código de ese monitor es el mismo.
- **Ruido de la VM:** la misma versión varió de 1.6 a 6.8 s entre corridas en una prueba previa. Diferencias menores a ~20% en archivos chicos **no son concluyentes**.

6. Plan de corrección sugerido

Prioridad	Acción	Hallazgo
1	Timeouts en el servidor y ciclo de <code>accept()</code> resistente a fallos	SEG-01, SEG-02
2	Longitud mínima de clave y TLS obligatorio por defecto	SEG-03
3	Validar rutas del manifiesto con <code>_ruta_segura</code>	SEG-04
4	Capturar <code>ValueError</code> por nombre y ocultar la clave en el prompt	SEG-05, SEG-06
5	Probar y ajustar <code>limpiar_terminal</code> en Windows	FUN-01
6	Opción de reanudación rápida (tamaño + fecha)	rendimiento
7	<code>chmod 0700</code> en carpetas de respaldo	SEG-07

7. Limitaciones de esta auditoría

- Todo se probó en **Linux, 1 vCPU, Python 3.12**. No probé Windows, macOS, Python 3.7/3.8, USB reales, MTP/iOS ni Android por `adb`.
- Los tiempos de copia sobre archivos grandes reflejan el caché de RAM, no discos físicos.
- El modo red se probó en `127.0.0.1`; no probé pérdida de paquetes ni latencia.
- No se revisó la versión GUI que comparte `config.json`.

